

ETL with DataFrames

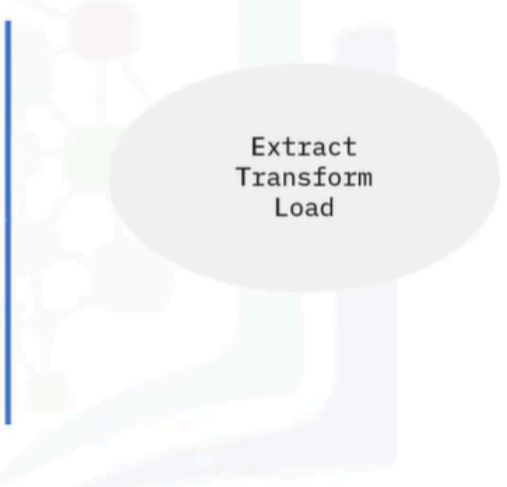
Sure! Let's talk about the basic operations of DataFrames, which are like tables that help us organize and analyze data.

Imagine you have a big box of LEGO bricks, each representing a piece of information, like the color, size, or shape of the bricks. A DataFrame is like a well-organized LEGO table where you can easily see and work with all your bricks. The basic operations you can perform with a DataFrame include reading the data (like sorting your LEGO bricks by color), analyzing it (checking how many of each color you have), transforming it (maybe you want to combine some bricks to make a new shape), and finally writing it back to a box or another table (storing your creations).

These operations help you manage and make sense of your data, just like organizing your LEGO collection helps you build amazing structures!

Basic DataFrame operations

- Read the data
- Analyze the data
- Transform the data
- Load data into a database
- Write data back to disk

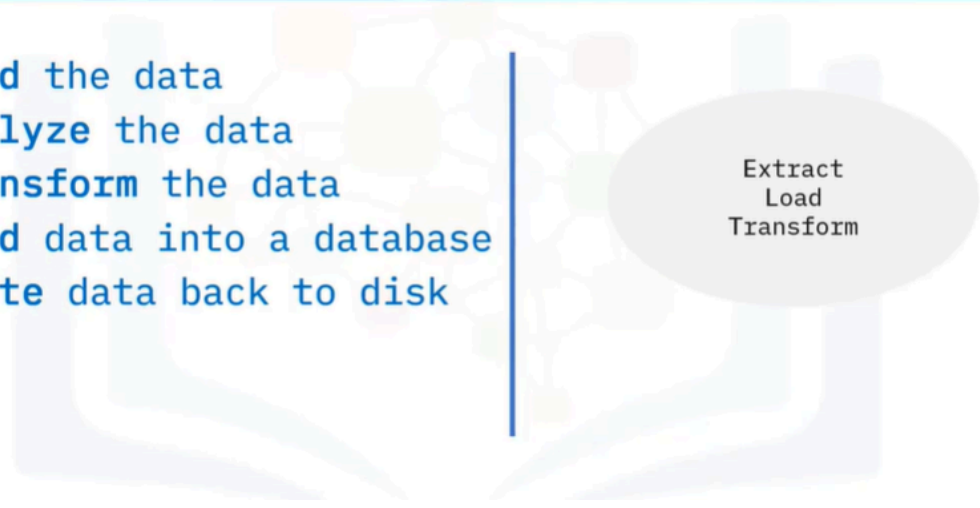


The diagram illustrates the ETL (Extract, Transform, Load) process. It features a vertical line on the left side of a light blue oval. To the right of the oval, the words 'Extract', 'Transform', and 'Load' are stacked vertically. The background of the slide is decorated with faint, colorful geometric shapes like hexagons and triangles.

Extract
Transform
Load

Basic DataFrame operations

- Read the data
- Analyze the data
- Transform the data
- Load data into a database
- Write data back to disk



The diagram features a light blue background with faint, overlapping hexagonal shapes. A vertical blue line separates the list of DataFrame operations on the left from a grey oval on the right. The oval contains the text 'Extract Load Transform'. The list of operations is positioned to the left of this line, and the oval is to the right.

Extract
Load
Transform

- Let's begin with basic DataFrame operations: Reading, Analysis, Transformation, Loading, and Writing. First, Spark reads in the data and loads the data into a DataFrame. Next, you can analyze the dataset, starting with simple tasks such as examining the columns, data types, number of rows, and optionally working with aggregated stats, trend analysis, and other operations. If needed, you can transform the data by filtering the data to locate specific values, sorting the dataset based on specific criteria, or by joining this dataset with another dataset. Then you can load your transformed dataset back to a database or a file system and write the data back to disk. This process is commonly also referred to as ETL – Extract, Transform, Load. ETL is an essential process in any data processing pipeline as the first step makes data accessible to data warehouses for machine learning models, downstream applications, or other services.
- Let's talk about another standard process. ELT – Extract, Load, and Transform. This process emerged as a result of big data processing. With ELT, all of the data resides in a data lake. A data lake is a vast pool of raw data, for which the purpose of the data is not pre-defined. When a data lake exists, each project forms individual transformation tasks as required, in contrast to anticipating all necessary transformation requirements usage scenarios as done with ETL and

a data warehouse. Organizations often use a mixture between both ETL and ELT.

Read the data

- Create a DataFrame
- Create a DataFrame from an existing DataFrame

```
import pandas as pd
mtcars = pd.read_csv('mtcars.csv')
sdf = spark.createDataFrame(mtcars)
```

Read the data

View the dataset parameters

```
[, 1] mpg Miles/(US) --> gallon
[, 2] cyl -> Number of cylinders
[, 3] disp --> Displacement (cu.in.)
[, 4] hp -> Gross horsepower
[, 5] drat -> Rear axle ratio
[, 6] wt -> Weight (lb/1000)
[, 7] qsec -> 1/4 mile time
[, 8] vs -> V/S
[, 9] am -> Transmission (0 = automatic, 1 = manual)
[,10] gear -> Number of forward gears
[,11] carb -> Number of carburetors
```

Read the data

View the first few dataset rows

	Unnamed: 0	mpg	cyl	displacement	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.9	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.6	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet SportAbout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

- Let's take a closer look at each basic DataFrame operation. When reading the data, you can directly load data into DataFrames or create a new Spark DataFrame from an existing DataFrame.
- The displayed code sample shows how to load a dataset into a Pandas DataFrame in Python and then load that same dataset into a Spark DataFrame object. Let's continue the example that uses the dataset about car models and performance.
- This dataset contains columns for parameters such as miles per gallon, number of cylinders, horsepower, and other auto-related parameters. Here you can view the first few rows of the dataset.

Analyze the data using printschema

View the schema

```
sdf.printSchema()
```

```
root
|-- Unnamed: 0: string (nullable = true)
|-- mpg: double (nullable = true)
|-- cyl: long (nullable = true)
|-- disp: double (nullable = true)
|-- hp: long (nullable = true)
|-- drat: double (nullable = true)
|-- wt: double (nullable = true)
|-- qsec: double (nullable = true)
|-- vs: long (nullable = true)
|-- am: long (nullable = true)
|-- gear: long (nullable = true)
|-- carb: long (nullable = true)
```

Analyze the data – the show function

Apply the show() function

```
sdf.show(5)
```

Unnamed: 0	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.9	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.6	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet SportAbout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

Analyze the data - the select function

Apply the `select()` function to view a specific column

```
sdf.select('mpg').show(5)
```

```
+----+  
| mpg |  
+----+  
| 21.0 |  
| 21.0 |  
| 22.8 |  
| 21.4 |  
| 18.7 |  
+----+  
only showing top 5  
rows
```

- Next is analyzing the data. First, examine the DataFrame column data types or the schema, using the print schema method. Take note of each column's data types. For example, the miles per gallon is a "double" type, while the first column indicates the car model name is a string. You can also examine a specified number of DataFrame using the show function. You can apply the select function to examine data from a specific column in detail. This example applies the select function to view the first five rows of the "mpg" column.

Transform the data - guidelines

- Keep only the relevant data
- Apply filters, joins, sources and tables, column operations, grouping and aggregations and other functions
- Apply domain-specific data augmentations processes

- After analyzing the dataset, your next step is to plan for any needed dataset transformations. The goal of a transformation is to retain only relevant data. Transformation techniques can include filtering the data, joining with other data sources, or performing columnar operations. Columnar operations are actions such as multiplying each column by a specific number, converting data from one unit to another, such as converting miles per gallon to kilometers per liter. Another common transformation technique is to group or aggregate data.
- Sometimes a downstream application may require aggregated data, such as a quarterly sales report that does not need to display every transaction but needs to show the aggregated total quarterly sales and revenue numbers. Many transformations are domain-specific data augmentation processes. The effort needed can vary depending on the domain and the data. For example, scientific datasets involving multiple, differing measurements may require more columnar operations and unit conversions, while financial data can require more aggregation and averaging tasks.

Transform the data using a filter

```
sdf.filter(sdf['mpg'] < 18).show(5)
```

Unnamed: 0	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Duster	14.3	8	160	110	3.90	2.620	16.46	0	1	4	4
Merc 280C	17.8	6	160	110	3.90	2.875	17.02	0	1	4	4
Merc 450SE	16.4	8	108.9	93	3.85	2.320	18.61	1	1	4	1
Merc 450SL	17.3	8	258.0	110	3.08	3.215	19.44	1	0	3	1
Merc 450SLC	15.2	8	360.0	175	3.15	3.440	17.02	0	0	3	2

- Here are some simple examples of transformations on the mtcars dataset. The filter function helps locate specific criteria within the dataset. In this case, the

task is to filter the dataset for cars that have a mileage, or mpg, of less than 18 miles per gallon.

Transform data using aggregation

```
car_counts = sdf.groupby(['cyl']).agg({"wt": "count"})\ .sort("count(wt)",  
ascending=False).show(5)
```

cyl	count(wt)
8	14
4	11
6	7

- Here is an example of aggregating data. In this example, the transformation aggregates all the records with the same number of cylinders, as seen in the "cyl" column, counts the number of vehicles with the specified number of cylinders, and sorts the data to list the number of units for each cylinder category in ascending order. We can see that 14 cars have an 8-cylinder engine and 11 cars have a 4-cylinder engine.

Loading or Exporting the Data

Is the final step of ETL pipeline

- Export to another database
- Export to disk as JSON files
- Save the data to a Postgres database
- Use an API to export data

- The ETL pipeline's last step is to export the data to disk or load it into another database. Like reading the dataset and extracting the data, you can write the data to the disk as a JSON file or save the data into another database such as a Postgres database. You can also use an API to export data to another database, such as a Postgres database.

Summary

In this video, you learned that:

- Basic DataFrame operations are reading, analysis, transformation, loading and writing
- You can use a Pandas DataFrame in Python to load a dataset
- You can apply the `printschema`, `select` function or `show` function for data analysis
- For transform tasks, keep only relevant data and apply functions such as filters, joins, column operations, grouping and aggregations and other functions